CUDA Profiling Report

Marcus Velazquez

1 Overview

This report summarizes CUDA device information and profiling results for three programs: `device_query.cu`, `vector_add.cu`, and `memory_stride.cu`. Measurements include device inventory, kernel timings, system-level profiling, and memory-access profiling.

Table 1: Programs used in the profiling exercises.

Program	**Purpose**
`device_query.cu`	Report GPU specifications and available memory.
`vector_add.cu`	Perform vector addition and report kernel execution time.
`memory_stride.cu`	Compare execution times for different memory strides.

2 Device Configuration and Inventory

The platform was configured in a 25 W NVIDIA power mode. The `tegrastats` command was used to observe GPU activity, with `GR3D_FREQ` approximately 0% when idle.

```
sudo nvpmodel -q
tegrastats --interval 500
./device_query
```

Table 2: Device information recorded from `device_query` and `Outputs.txt`.

Parameter	**Recorded value**
GPU	Orin
Streaming multiprocessors (SMs)	8
Warp size	32 threads
Maximum threads per block	1024
Shared memory per block	48 KB
GPU memory (recorded in `Outputs.txt`)	4345 / 7607 MB
NVIDIA power mode	25 W (mode 1)

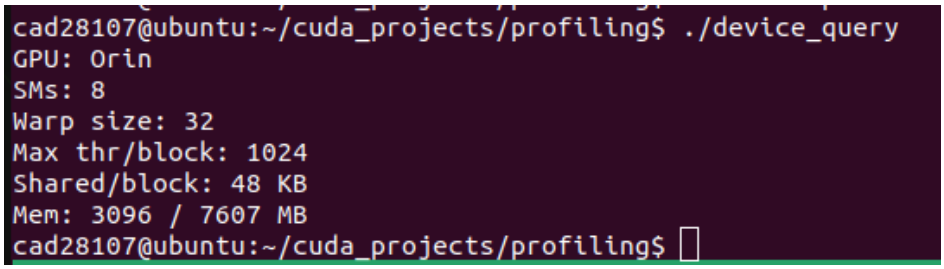


Figure 1: device_query terminal output

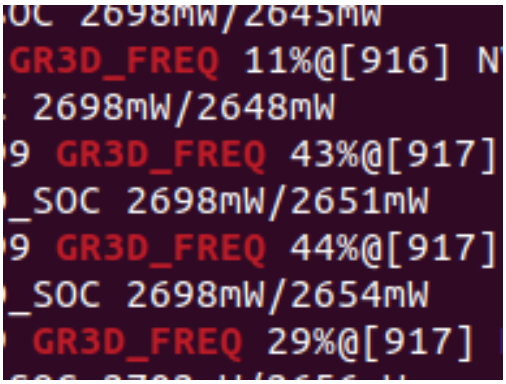


Figure 2: GR3D_FREQ under usage.

3 Vector Addition

The `vector_add.cu` program was compiled and executed to record a kernel time and verify the output value `c[100] = 300.000000`.

```
nvcc --ptxas-options=-v add_vector.cu -o add_vector
./add_vector
```

Table 3: Vector addition timing and output checks.

Measurement	**Recorded value**
Kernel time (initial run screenshot)	0.889504 ms
Kernel time (`Outputs.txt` run)	0.882912 ms
Output check, `c[100]`	300.000000

Table 4: Compiler resource information for `vecAdd`.

Compiler-reported parameter	Value
Compiled architecture in log	`sm_52`
Global memory (`gmem`)	0 bytes
Stack frame	0 bytes
Spill stores / spill loads	0 bytes / 0 bytes
Registers used	8
Barriers used	0
Constant memory (`cmem[0]`)	348 bytes

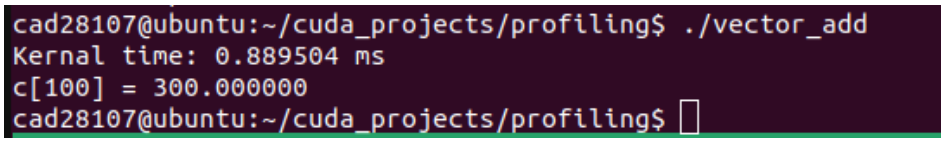


Figure 3: vector_add terminal output.

3.1 Nsight Systems Results

The system profiler output was saved in `nsys_test.txt`. The reported kernel and transfer timings are summarized below.

Table 5: Nsight Systems summary for vector addition.

Profiler summary	Operation	Reported time
`cuda_gpu_kern_sum`	`vecAdd(...)`	0.18 ms
`cuda_gpu_mem_time_sum`	Host to device (HtoD)	0.95 ms
`cuda_gpu_mem_time_sum`	Device to host (DtoH)	1.41 ms
`cuda_api_sum`	`cudaMalloc`	73.4 ms

4 Memory Stride Experiment

The `memory_stride.cu` program performs memory accesses at two different strides. A warm-up launch is included before the measured launches.

```
./memory_stride
```

Table 6: Memory stride timings from program executions.

Execution	Stride 1	Stride 32
Initial run screenshot	0.537120 ms	5.889504 ms
`Outputs.txt` run	0.538240 ms	6.034272 ms

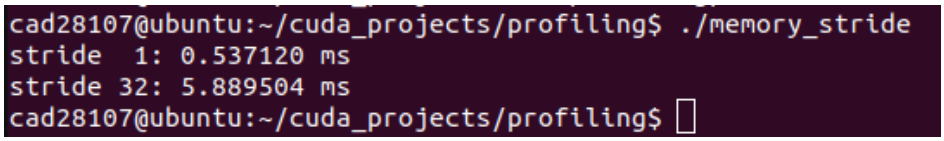


Figure 4: memory_stride terminal output.

4.1 Nsight Compute Results

The following command was used to collect basic kernel metrics:

```
sudo /usr/local/cuda-12.6/bin/ncu --set basic ./memory_stride > ncu_test.txt
```

The profiler output identifies `copyStride - 0` as the stride-1 warm-up, `copyStride - 1` as the measured stride-1 launch, and `copyStride - 2` as the measured stride-32 launch.

Table 7: Nsight Compute profiling comparison.

Metric	Stride 1	Stride 32
Duration	0.644 ms	5.85 ms
Memory throughput (%)	49.1	70.34
Achieved occupancy (%)	83.77	89.89
Sectors per request	4	32

5 Observations

The stride-32 kernel required substantially more time than the stride-1 kernel in both the program timing measurements and the Nsight Compute results. Nsight Compute also reported a larger number of sectors per request for stride 32 (32 versus 4), consistent with less efficient memory access across neighboring threads. The recorded memory throughput and achieved occupancy percentages were higher for the stride-32 case despite its longer execution time.

The Nsight Systems results indicate that GPU memory transfers accounted for substantially more execution time than the vector addition kernel itself. The kernel required approximately 0.178 ms, while host-to-device and device-to-host transfers required a combined 2.364 ms. Additionally, the CUDA API summary reported 73.421 ms spent in cudaMalloc, highlighting the significant overhead associated with GPU memory allocation.

5.1 Two Sentences

1. Vector addition's execution time was primarily spent on GPU memory allocation and host-to-device/device-to-host transfers rather than the kernel itself, which required only 0.178 ms compared to 73.421 ms for cudaMalloc and 2.364 ms for GPU memory transfers.

2. Stride 32 is slower because neighboring threads access memory locations farther apart, making memory access less efficient and requiring more memory operations, even though the instructions are the same.

6 Appendix

6.1 device_query.cu

```cpp
// device_query.cu

#include <stdio.h>
#include <cuda_runtime.h>

int main()
{
    cudaDeviceProp prop;
    cudaGetDeviceProperties(&prop, 0);

    printf("GPU: %s\n", prop.name);
    printf("SMs: %d\n", prop.multiProcessorCount);
    printf("Warp size: %d\n", prop.warpSize);
    printf("Max thr/block: %d\n", prop.maxThreadsPerBlock);
    printf("Shared/block: %zu KB\n", prop.sharedMemPerBlock/1024);

    size_t free_b, total_b;
    cudaMemGetInfo(&free_b, &total_b);
    printf("Mem: %zu / %zu MB\n",
           free_b>>20, total_b>>20);

    return 0;
}
```

Listing 1: CUDA Device Query Implementation

6.2 add_vector.cu

```cpp
// add_vector.cu

#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void vecAdd(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
```

```
}

int main() {
    int n = 1 << 20; // 1,048,576
    size_t size = n * sizeof(float);

    // 1. CPU Arrays
    float *h_a = (float*)malloc(size);
    float *h_b = (float*)malloc(size);
    float *h_c = (float*)malloc(size);

    for (int i = 0; i < n; i++)
        { h_a[i] = i; h_b[i] = 2*i; }

    // 2. GPU arrays
    float *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, size);
    cudaMalloc(&d_b, size);
    cudaMalloc(&d_c, size);

    // 3. Copy inputs across the bridge
    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

    // 4. Time the kernel with events
    // (2nd profiling technique)
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    vecAdd<<< (n+255) / 256, 256>>>
           (d_a, d_b, d_c, n);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float ms;
    cudaEventElapsedTime(&ms, start, stop);
    printf("Kernal time: %f ms\n", ms);

    // 5. Copy the result back
    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

    // Verify: 100 + 200 = 300
    printf("c[100] = %f\n", h_c[100]);

    // 6. Free both worlds
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);

    free(h_a);
    free(h_b);
    free(h_c);

    return 0;
}
```

Listing 2: CUDA Vector Addition Implementation

6.3 memory_stride.cu

```cpp
// memory_stride.cu

#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void copyStride(float *in, float *out, int n, int stride)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = (long long)i * stride % n;

    if (i < n)
        out[j] = in[j];
}

/*
 stride 1: thread i touches j = i
    neighbors read neighbors

 stride 32: j jumps by 32 floats
    every thread hits its own memory segment
*/

int main() {
    int n = 1 << 22;
    size_t size = n * sizeof(float);

    float *d_in, *d_out;
    cudaMalloc(&d_in, size);
    cudaMalloc(&d_out, size);

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Warm-up: first launch pays
    // one-time setup -> never time it!
    copyStride<<<(n+255)/256, 256>>>
             (d_in, d_out, n, 1);

    for (int s = 1; s <= 32; s *= 32)
    {
        cudaEventRecord(start);

        copyStride<<<(n+255)/256, 256>>>
                 (d_in, d_out, n, s);

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);

        float ms;
        cudaEventElapsedTime(&ms, start, stop);

        printf("stride %2d: %f ms\n", s, ms);
    }

    return 0;
```

```
58 }
```

Listing 3: CUDA Memory Stride Implementation